

Practice Questions

1. Write a `hash_password()` and `verify_password()` utility using `passlib`. Test them in isolation before wiring into routes.

```
Original Password: mypassword12345
Hashed Password: $2b$12$30e4VfnnCh0wrz27wk1VoeYq26xdTmcyYUy9qe3eImksJVq8/a2ou
Correct Password: True
Wrong Password: False
```

Week 2 > Day 4 > main.py > ...

```
1  from security import hash_password, verify_password
2
3
4  password='mypassword12345'
5  hashed=hash_password(password)
6  print(f"Original Password: {password}")
7  print(f"Hashed Password: {hashed}")
8  print(f"Correct Password: {verify_password(password, hashed)}")
9  print(f"Wrong Password: {verify_password('wrongpassword', hashed)}")
```

2. Create a POST `/auth/register` route that accepts username + password, hashes the password, and saves the user.

Authentication

POST /auth/register Register

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "username": "Alina",
  "password": "Alina1234"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/auth/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "Alina",
    "password": "Alina1234"
  }'
```

Request URL

```
http://127.0.0.1:8000/auth/register
```

Server response

Code	Details
200	Response body<pre>{ "message": "User registered successfully" }</pre>Download Response headers<pre>content-length: 42 content-type: application/json date: Fri, 10 Jul 2026 14:24:25 GMT server: uvicorn</pre>

Response

Code	Description	Links
------	-------------	-------

3. Create a POST /auth/token route using OAuth2PasswordRequestForm that verifies credentials and returns a signed JWT.

POST /auth/token Login

Parameters

CancelReset

No parameters

Request body required

application/x-www-form-urlencoded

grant_type

string | (string | null)
pattern: ^password\$

grant_type

☒ Send empty value

username required

string

Alina

password required

string(\$password)

scope

string

scope

☒ Send empty value

client_id

string | (string | null)

string

☐ Send empty value

client_secret

string | (string | null)(\$password)

☐ Send empty value

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/auth/token' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=&username=Alina%26&password=alinal234&scope=&client_id=string&client_secret=*****'
```

Request URL

http://127.0.0.1:8080/auth/token

Server response

Code

Details

401

Undocumented

Error: Unauthorized

Response body

```
{
  "detail": "Invalid credentials"
}
```

Download

Response headers

```
content-length: 32
content-type: application/json
date: Fri, 19 Jul 2026 14:34:05 GMT
server: uvicorn
```

Parameters

Cancel

Reset

No parameters

Request body required application/x-www-form-urlencoded

grant_type

string | (string | null)

pattern: "password\$"

password

☐ Send empty value

username required

string

Ali

password required

string(\$password)

scope

string

scope

☒ Send empty value

client_id

string | (string | null)

string

☐ Send empty value

client_secret

string | (string | null)(\$password)

☐ Send empty value

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/auth/token' \
-H 'accept: application/json' \
-H 'Content-Type: application/x-www-form-urlencoded' \
-d 'grant_type=password&username=Ali&password=123&scope=&client_id=string&client_secret=*****'
```

Request URL

http://127.0.0.1:8000/auth/token

Server response

Code

Details

200

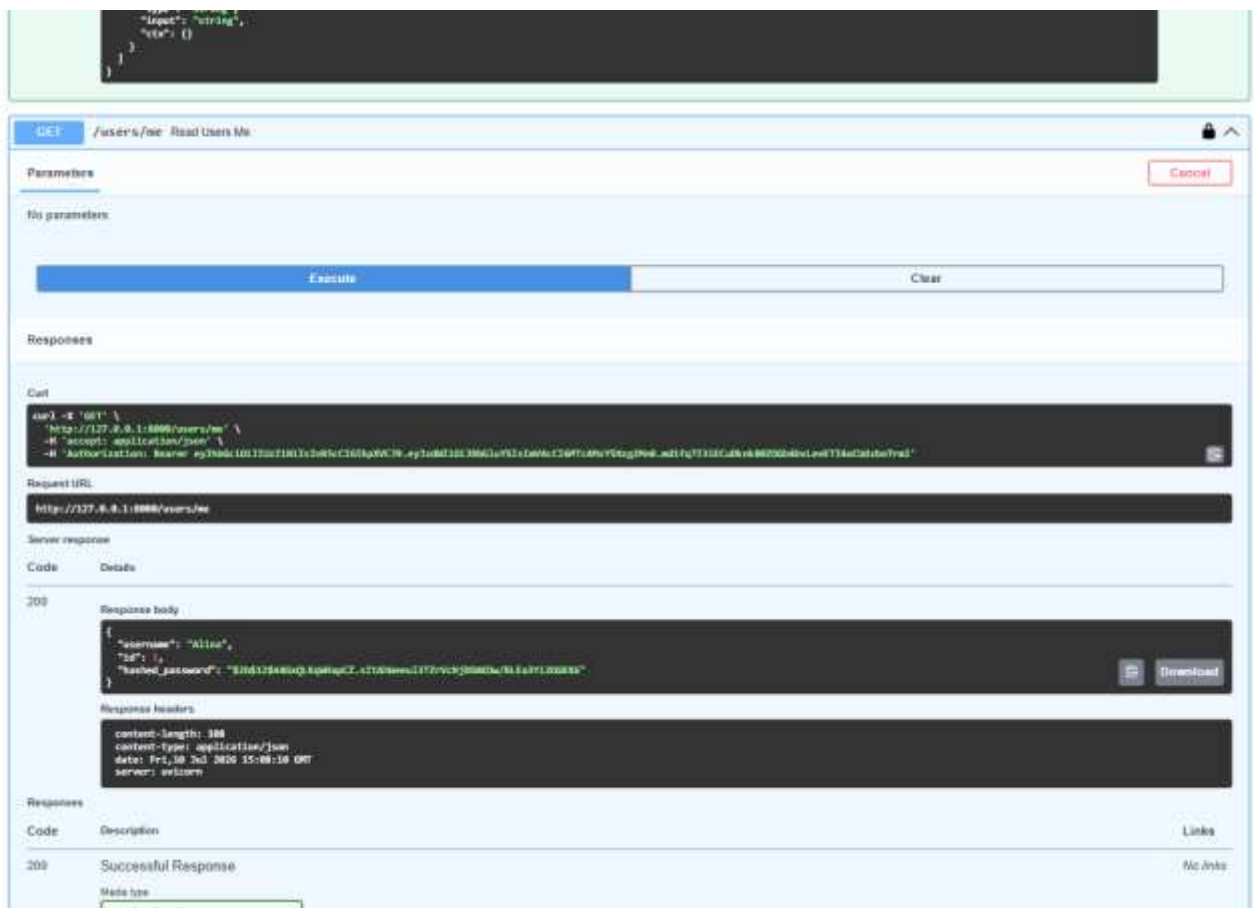
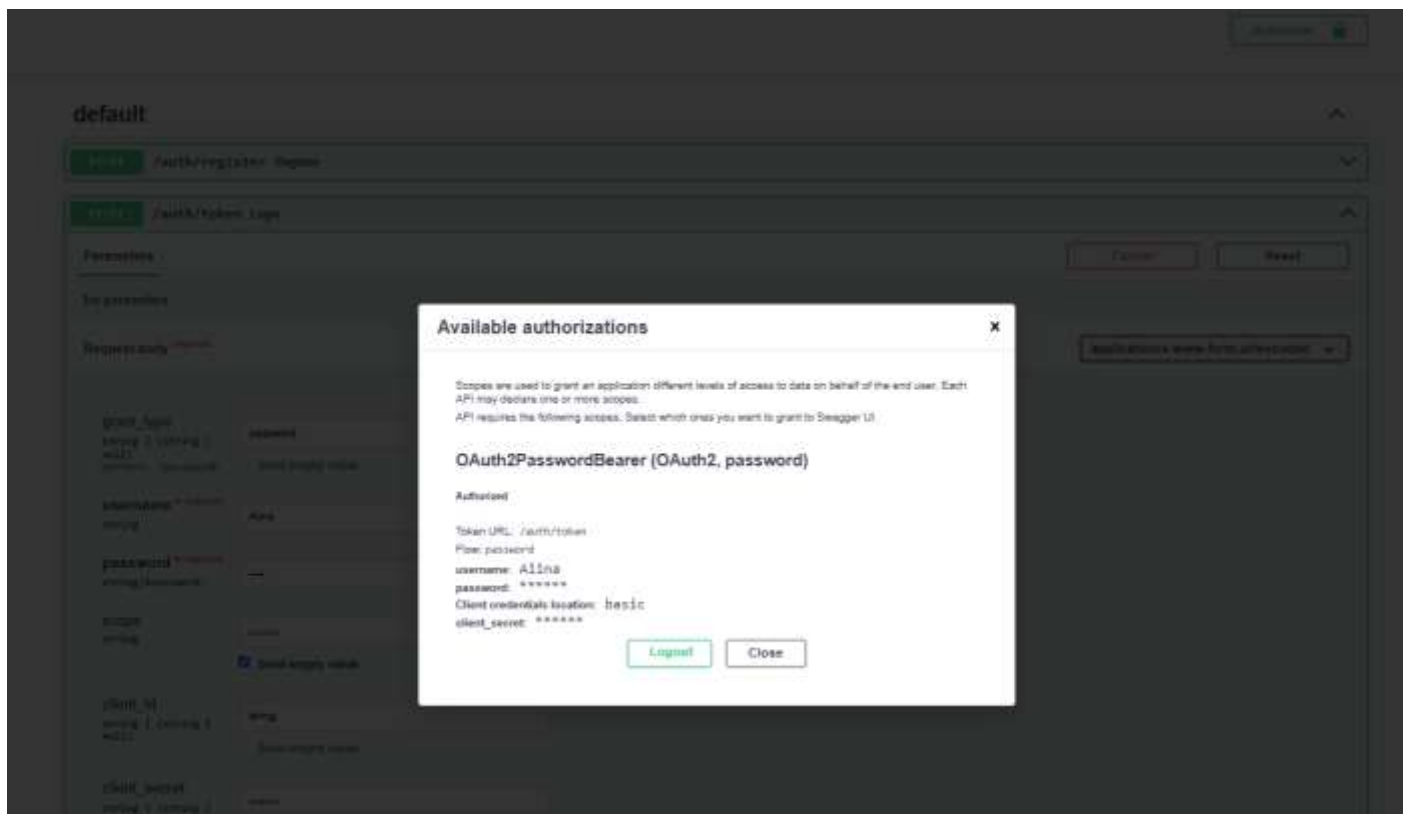
Response body

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI3bGk1C3leHA10jE3ODM2OTYwMTB9.CrFw33z9s1gVD2zn3yg4FM4Hx03vz0NgVYs00f-zIZQ",
  "token_type": "bearer"
}
```

Response headers

```
content-length: 162
content-type: application/json
date: Fri, 10 Jul 2026 14:36:48 GMT
server: uvicorn
```

- Write a `get_current_user` dependency that reads the `Authorization: Bearer <token>` header, decodes the JWT, and returns the user or raises 401.



6. Add a role field to your user model. Write a require_admin dependency that raises 403 if the current user is not an admin, and apply it to a DELETE /users/{id} route.

When Simple user logged in:

s#/default/read_users_me_users_me_get

DELETE /users/{id} Delete User

Parameters

Cancel

Name	Description
id * required integer (path)	1

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8080/users/1' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bmR1b3R5b29kaW4iOiJ0dG90OTgwIiwiaWF0IjoxNDQ0MzUyOTUwLCJ1aW4iOiJ1b3R5b29kaW4ifQ'
```

Request URL

```
http://127.0.0.1:8080/users/1
```

Server response

Code	Details
403 Unauthorized	Error: Forbidden Response body <pre>{ "detail": "Access only" }</pre> Download Response headers <pre>content-length: 34 content-type: application/json date: Fri, 18 Jul 2025 15:17:48 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links